



**WYDZIAŁ
ELEKTROTECHNIKI
I INFORMATYKI**
POLITECHNIKI RZESZOWSKIEJ

Cyberbezpieczeństwo

Projekt

Temat projektu

Szyfrowanie plików i bezpieczna wymiana kluczy

(alt: System bezpiecznej wymiany plików z wykorzystaniem kryptografii symetrycznej i asymetrycznej)

Jakub Lewandowski
3EF-ZI
178746

i

Paweł Torba
3EF-ZI
178763

Spis treści

1. Cel projektu.....	4
2. Główne funkcje i zakres systemu.....	4
2.1. Szyfrowanie danych.....	4
2.2. Deszyfrowanie danych.....	4
2.3. Tworzenie podpisu cyfrowego.....	4
2.4. Weryfikacja podpisu cyfrowego.....	4
2.5. Generowanie kluczy.....	4
2.6. Szyfrowanie hybrydowe.....	5
3. Podstawy teoretyczne kryptografii.....	5
3.1. Kryptografia symetryczna.....	5
3.2. Kryptografia asymetryczna.....	6
3.3. Podpis cyfrowy.....	7
3.4. Integralność i poufność danych.....	7
3.5. Algorytm AES.....	7
3.5.1. Proces szyfrowania.....	8
3.5.2. Rozszerzanie klucza (Key Expansion).....	8
3.5.3. Zalety algorytmu AES.....	9
3.5.4. Wady algorytmu AES.....	9
3.5.5. Zastosowania algorytmu AES.....	9
3.6. Algorytm RSA.....	9
3.6.1. Zasada działania RSA.....	10
3.6.2. Podpisywanie wiadomości.....	10
3.6.3. Zastosowanie.....	10
3.6.4. Zalety RSA.....	11
3.6.5. Wady RSA.....	11
3.7. SHA-256.....	11
3.7.1. Proces obliczania skrótu.....	11
3.7.2. Zalety algorytmu SHA-256.....	12
3.7.3. Wady algorytmu SHA-256.....	12
3.7.4. Zastosowania.....	12
4. Technologie i narzędzia.....	12
5. Architektura aplikacji.....	13
5.1. Struktura katalogów i plików.....	13
5.2. Moduły systemu.....	14
5.2.1. Moduł szyfrowania.....	14
5.2.2. Moduł deszyfrowania.....	17
5.2.3. Moduł podpisu cyfrowego.....	21
5.2.5. Moduł generowania kluczy.....	23
6. Bezpieczeństwo systemu i możliwe zagrożenia.....	25
6.1. Założenia bezpieczeństwa systemu.....	25
6.2. Ochrona poufności danych.....	26
6.3. Bezpieczna wymiana kluczy.....	26
6.4. Integralność i autentyczność danych.....	26
6.5. Możliwe zagrożenia.....	26
6.7. Metody ograniczania ryzyka.....	27
7. Możliwości wdrożenia i zastosowania systemu.....	28

7.1. Zastosowanie systemu i przykładowa wymiana plików.....	28
7.2. Ograniczenia obecnej wersji systemu.....	28
7.3. Możliwości dalszego rozwoju.....	29
8. Przykłady użycia i testowanie systemu.....	29
8.1. Generowanie kluczy RSA.....	29
8.2. Szyfrowanie i odszyfrowanie dokumentu.....	30
8.3. Szyfrowanie hybrydowe.....	32
8.4. Podpisanie dokumentu i weryfikacja podpisu.....	34
8.5. Kompatybilność z innymi programami.....	35
8.6. Ocena poprawności działania systemu.....	37
9. Podsumowanie i wnioski.....	38
10. Bibliografia.....	39

1. Cel projektu

Celem projektu jest opracowanie prostej aplikacji umożliwiającej bezpieczne szyfrowanie i wymianę danych. System ma pozwalać na zaszyfrowanie pliku oraz tekstu, generowanie klucza szyfrującego (i jego szyfrowanie w celu bezpiecznej wymiany), podpisanie pliku podpisem cyfrowym, weryfikację podpisu oraz odszyfrowanie pliku po stronie odbiorcy.

Projekt obejmuje również analizę teoretycznych podstaw kryptografii, w szczególności zagadnień związanych z szyfrowaniem symetrycznym i asymetrycznym, podpisem cyfrowym, bezpieczną wymianą kluczy oraz mechanizmami zapewniania integralności i bezpieczeństwa danych.

2. Główne funkcje i zakres systemu

2.1. Szyfrowanie danych

Użytkownik może wskazać plik wejściowy, lokalizację pliku wyjściowego, algorytm szyfrowania oraz podstawową konfigurację zmiennych algorytmu szyfrującego. Aplikacja automatycznie generuje klucze (np.: *AES*) oraz wartość inne konieczne wartości (np.: *Nonce/IV*), wymagane do bezpiecznego zaszyfrowania pliku.

2.2. Deszyfrowanie danych

Użytkownik może wskazać zaszyfrowany plik, lokalizację pliku wynikowego, klucz oraz inne wartość wykorzystane podczas szyfrowania. W przypadku zastosowania szyfrowania hybrydowego możliwe jest również wykorzystanie klucza prywatnego do odszyfrowania klucza szyfrującego.

2.3. Tworzenie podpisu cyfrowego

Użytkownik może wskazać plik przeznaczony do podpisania, klucz prywatny oraz lokalizację zapisu podpisu cyfrowego. Aplikacja generuje skrót pliku przy użyciu algorytmu SHA-256, a następnie tworzy podpis cyfrowy, który pozwala potwierdzić autentyczność nadawcy oraz integralność danych.

2.4. Weryfikacja podpisu cyfrowego

Użytkownik może wskazać plik, podpis cyfrowy oraz klucz publiczny nadawcy. Aplikacja sprawdza poprawność podpisu i informuje, czy plik został podpisany przez właściciela wskazanego klucza prywatnego oraz czy jego zawartość nie została zmodyfikowana po podpisaniu.

2.5. Generowanie kluczy

Użytkownik może wygenerować parę kluczy (publiczny i prywatny), określając ich rozmiar oraz lokalizację zapisania klucza publicznego i prywatnego. Wygenerowane klucze mogą zostać wykorzystane do szyfrowania hybrydowego oraz tworzenia i weryfikacji podpisów cyfrowych.

2.6. Szyfrowanie hybrydowe

Użytkownik może zdecydować, czy wygenerowany podczas szyfrowania klucz ma zostać dodatkowo zabezpieczony przy użyciu klucza publicznego odbiorcy. W takim przypadku dane są szyfrowane wybranym algorytmem, natomiast klucz szyfrowania zostaje zaszyfrowany algorytmem RSA. Dzięki temu możliwe jest bezpieczne przekazanie klucza szyfrującego odbiorcy. Rozwiązanie to łączy wysoką wydajność kryptografii symetrycznej z bezpieczeństwem wymiany kluczy oferowanym przez kryptografię asymetryczną.

3. Podstawy teoretyczne kryptografii

Kryptografia to praktyka i nauka zajmująca się technikami bezpiecznej komunikacji w obliczu działań podmiotów o wrogich zamiarach. Kryptografia polega na tworzeniu i analizowaniu protokołów, które uniemożliwiają osobom trzecim lub ogółowi społeczeństwa odczytanie prywatnych wiadomości.

Współczesna kryptografia stanowi punkt styku takich dyscyplin, jak matematyka, informatyka, bezpieczeństwo informacji, elektrotechnika, cyfrowe przetwarzanie sygnałów czy fizyka. Podstawowe pojęcia związane z bezpieczeństwem informacji (poufność danych, integralność danych, uwierzytelnianie i niezaprzeczalność) są również kluczowe dla kryptografii. Praktyczne zastosowania kryptografii obejmują handel elektroniczny, karty płatnicze oparte na chipach, waluty cyfrowe, hasła komputerowe oraz komunikację wojskową.

3.1. Kryptografia symetryczna

Kryptografia symetryczna polega na tym, że nadawca i odbiorca używają tego samego tajnego klucza do szyfrowania i odszyfrowywania danych (klucz symetryczny) - oznacza to, że zarówno nadawca, jak i odbiorca muszą posiadać ten sam tajny klucz.

Szyfry oparte o klucz symetrycznym są realizowane albo jako szyfry blokowe, albo jako szyfry strumieniowe. Szyfr blokowy szyfruje dane wejściowe w postaci bloków tekstu jawnego, w przeciwieństwie do pojedynczych znaków, które są formą wejściową stosowaną w szyfrach strumieniowych (bit po bicie lub znak po znaku). Do szyfrów blokowych zaliczają się m.in.: *DES* (*Data Encryption Standard*) czy *AES* (*Advanced Encryption Standard*). Do szyfrów strumieniowych zalicza się *RC4*.

Zalety kryptografii symetrycznej:

- wysoka szybkość działań,
- niskie wymagania obliczeniowe,
- wysoka wydajność.

Wady kryptografii symetrycznej:

- problem dystrybucji klucza,
- skalowalność.

3.2. Kryptografia asymetryczna

Kryptografia asymetryczna polega na wykorzystaniu dwóch różnych, lecz matematycznie powiązanych kluczy: klucza publicznego i klucza prywatnego. Klucz publiczny może być udostępniany wszystkim użytkownikom, natomiast klucz prywatny musi pozostać tajny i być znany wyłącznie właścicielowi. Dane zaszyfrowane kluczem publicznym mogą zostać odszyfrowane jedynie odpowiadającym mu kluczem prywatnym.

Do najpopularniejszych algorytmów asymetrycznych zalicza się:

- *RSA (Rivest–Shamir–Adleman)*,
- *Diffie–Hellman*,
- *ECC (Elliptic Curve Cryptography)*,
- *DSA (Digital Signature Algorithm)*.

Kryptografia asymetryczna jest wykorzystywana nie tylko do szyfrowania danych, ale również do tworzenia podpisów cyfrowych, które umożliwiają potwierdzenie autentyczności nadawcy oraz integralności przesyłanych danych.

Szyfry oparte o klucz asymetrycznym są realizowane albo jako szyfry blokowe, albo jako szyfry strumieniowe. Szyfr blokowy szyfruje dane wejściowe w postaci bloków tekstu jawnego, w przeciwieństwie do pojedynczych znaków, które są formą wejściową stosowaną w szyfrach strumieniowych (bit po bicie lub znak po znaku). Do szyfrów blokowych zaliczają się m.in.: *DES (Data Encryption Standard)* czy *AES (Advanced Encryption Standard)*. Do szyfrów strumieniowych zalicza się *RC4*.

Zalety kryptografii asymetrycznej:

- brak konieczności bezpiecznego przekazywania tajnego klucza,
- możliwość stosowania podpisów cyfrowych,
- wysoki poziom bezpieczeństwa,
- dobra skalowalność w dużych sieciach.

Wady kryptografii asymetrycznej:

- mniejsza szybkość działania w porównaniu z kryptografią symetryczną,
- większe wymagania obliczeniowe,
- nieefektywność przy szyfrowaniu dużych ilości danych.

W praktyce kryptografia asymetryczna jest często łączona z kryptografią symetryczną. Klucz publiczny służy do bezpiecznej wymiany klucza symetrycznego, natomiast właściwe dane są szyfrowane szybszym algorytmem symetrycznym, takim jak *AES*.

3.3. Podpis cyfrowy

Podpis cyfrowy to mechanizm służący do potwierdzania autentyczności i integralności danych. Jest on tworzony przy użyciu klucza prywatnego nadawcy, a weryfikowany za pomocą klucza publicznego.

Funkcje podpisu cyfrowego:

- potwierdzenie tożsamości nadawcy,
- ochrona przed modyfikacją danych,
- brak możliwości wyparcia się podpisu (niezaprzeczalność).

Podpis cyfrowy często wykorzystuje funkcje skrótu (np. *SHA-256*) oraz algorytmy asymetryczne (np. *RSA*).

3.4. Integralność i poufność danych

Integralność danych oznacza pewność, że dane nie zostały zmienione w nieautoryzowany sposób podczas przesyłania lub przechowywania. Do jej zapewnienia stosuje się m.in. funkcje skrótu i podpisy cyfrowe.

Poufność danych oznacza ochronę informacji przed nieautoryzowanym dostępem. Zapewnia się ją głównie poprzez szyfrowanie danych (np. *AES* lub *RSA*).

Obie te właściwości są fundamentem bezpieczeństwa systemów informatycznych i komunikacji cyfrowej.

3.5. Algorytm AES

AES (Advanced Encryption Standard) należy do algorytmów szyfrowania symetrycznego. W 2001 roku został przyjęty przez amerykański instytut *National Institute of Standards and Technology* jako oficjalny standard szyfrowania.

AES jest szyfrem blokowym, co oznacza, że szyfruje dane w blokach o stałej długości 128 bitów. Algorytm może wykorzystywać klucze o długości:

- 128 bitów,
- 192 bitów,
- 256 bitów.

3.5.1. Proces szyfrowania

Proces szyfrowania w AES składa się z kilku etapów:

1. *AddRoundKey* (dodanie klucza rundy): jest to etap wykonywany na początku szyfrowania oraz na końcu każdej rundy. Każdy bajt danych jest łączony z odpowiednim bajtem klucza rundy za pomocą operacji *XOR*. Celem tego etapu jest wprowadzenie tajnego klucza do procesu szyfrowania.
2. *SubBytes* (podstawianie bajtów): każdy bajt stanu zostaje zastąpiony innym bajtem zgodnie ze tabelą *S-Box* (specjalna tablica podstawień, jej zadaniem jest zastąpienie każdego bajtu inną wartością według wcześniej zdefiniowanej reguły), w celu zwiększenia nieliniowości algorytmu i utrudnienia analiz kryptograficznych. Zamiana odbywa się według wcześniej zdefiniowanej tabeli.
3. *ShiftRows* (przesunięcie wierszy): wiersze macierzy stanu są cyklicznie przesuwane w lewo.
4. *MixColumns* (mieszanie kolumn): każda kolumna stanu jest przekształcana za pomocą operacji matematycznych wykonywanych w skończonym ciele *Galois $GF(2^8)$* . W praktyce bajty w obrębie kolumn są ze sobą mieszane, dzięki czemu zmiana jednego bajtu wpływa na wiele innych bajtów. Krok ten wykonywany jest w celu utrudnienia odtworzenia danych wejściowych.
5. *AddRoundKey* (ponowne dodanie klucza): wynik poprzednio wykonanych operacji jest ponownie łączony z kluczem rundy za pomocą operacji *XOR*.
6. W ostatniej rundzie pomijany jest etap *MixColumns*.

3.5.2. Rozszerzanie klucza (Key Expansion)

Przed rozpoczęciem szyfrowania klucz główny jest przekształcany w zestaw kluczy rundowych. Dla *AES-128* z jednego klucza 128-bitowego generowanych jest 11 kluczy:

- 1 klucz początkowy,
- 10 kluczy dla kolejnych rund.

Dzięki temu każda runda wykorzystuje inny fragment klucza.

W zależności od długości klucza wykonywana jest różna liczba rund szyfrowania:

- *AES-128* – 10 rund,
- *AES-192* – 12 rund,
- *AES-256* – 14 rund.

3.5.3. Zalety algorytmu AES

- bardzo wysoki poziom bezpieczeństwa,
- szybkie działanie,
- niskie wymagania obliczeniowe,
- możliwość zastosowania w sprzęcie i oprogramowaniu,
- powszechne zastosowanie na całym świecie.

3.5.4. Wady algorytmu AES

- konieczność bezpiecznej wymiany klucza szyfrującego,
- bezpieczeństwo zależy od odpowiedniego zarządzania kluczami.

3.5.5. Zastosowania algorytmu AES

- szyfrowanie dysków i nośników danych,
- zabezpieczanie sieci Wi-Fi (WPA2, WPA3),
- połączenia VPN,
- bankowość elektroniczna,
- komunikatory internetowe,
- protokół *HTTPS/TLS*.

Obecnie *AES* jest uznawany za standard szyfrowania i jest szeroko stosowany przez instytucje rządowe, firmy oraz użytkowników indywidualnych do ochrony poufnych danych.

3.6. Algorytm RSA

RSA jest jednym z najważniejszych algorytmów kryptografii asymetrycznej. Jego bezpieczeństwo opiera się na trudności rozkładu dużych liczb na czynniki pierwsze.

RSA wykorzystuje parę kluczy:

- klucz publiczny – służy do szyfrowania danych lub weryfikacji podpisu cyfrowego,
- klucz prywatny – służy do odszyfrowywania danych lub tworzenia podpisu cyfrowego.

3.6.1. Zasada działania RSA

Proces działania algorytmu RSA składa się z kilku etapów:

1. Generowanie kluczy: w celu wygenerowania pary kluczy (publicznego i prywatnego) należy wykonać następujące kroki:
 1. wybrać dwie duże liczby pierwsze „ p ” oraz „ q ”,
 2. obliczyć wartość $n = p \cdot q$,
 3. obliczyć funkcję Eulera $\varphi(n) = (p-1)(q-1)$,
 4. wybrać liczbę „ e ”, która jest względnie pierwsza z $\varphi(n)$,
 5. wyznaczyć liczbę „ d ”, będącą odwrotnością modularną liczby „ e ”, tak aby spełniona była zależność: $d \cdot e \equiv 1 \pmod{\varphi(n)}$

Klucz publiczny definiowany jest jako para (n, e) , natomiast klucz prywatny jako para (n, d) .

2. Szyfrowanie wiadomości: przed zaszyfrowaniem wiadomość dzielona jest na bloki o wartości liczbowej nie większej niż „ n ”. Następnie każdy blok wiadomości „ m ” szyfrowany jest przy użyciu klucza publicznego według wzoru: $c = m^e \pmod{n}$. W wyniku powstaje szyfrogram składający się z kolejnych bloków „ c ”.
3. Deszyfrowanie wiadomości: odbiorca wykorzystuje swój klucz prywatny do odszyfrowania otrzymanego szyfrogramu. Każdy blok szyfrogramu „ c ” przekształcany jest z powrotem w wiadomość jawną „ m ” zgodnie ze wzorem: $m = c^d \pmod{n}$. Po odszyfrowaniu wszystkich bloków otrzymywana jest oryginalna wiadomość.

3.6.2. Podpisywanie wiadomości

Algorytm RSA może być również wykorzystywany do tworzenia podpisów cyfrowych. W tym celu nadawca oblicza skrót wiadomości, a następnie szyfruje go swoim kluczem prywatnym. Tak utworzony podpis jest przesyłany wraz z wiadomością.

Odbiorca wykorzystuje klucz publiczny nadawcy do odszyfrowania podpisu i porównuje otrzymaną wartość ze skrótem obliczonym z odebranej wiadomości. Jeżeli obie wartości są zgodne, oznacza to, że wiadomość pochodzi od deklarowanego nadawcy i nie została zmodyfikowana podczas transmisji.

3.6.3. Zastosowanie

Ze względu na wysoką złożoność obliczeniową RSA rzadko wykorzystuje się go do szyfrowania całych wiadomości. Najczęściej służy on do bezpiecznej wymiany kluczy szyfrujących lub tworzenia podpisów cyfrowych, natomiast właściwe dane szyfrowane są szybszymi algorytmami symetrycznymi, takimi jak AES.

3.6.4. Zalety RSA

- wysoki poziom bezpieczeństwa,
- szeroko stosowany i dobrze przebadany,
- umożliwia bezpieczną wymianę kluczy.

3.6.5. Wady RSA

- działa znacznie wolniej niż *AES*,
- wymaga dużych kluczy (najczęściej 2048 lub 4096 bitów),
- nie nadaje się do szyfrowania dużych ilości danych.

3.7. SHA-256

SHA-256 (Secure Hash Algorithm 256-bit) Jest kryptograficzną funkcją skrótu, której zadaniem jest przekształcenie danych wejściowych o dowolnej długości w skrót (*hash*) o stałej długości 256 bitów. Funkcja ta jest jednokierunkowa, co oznacza, że na podstawie otrzymanego skrótu nie można odtworzyć oryginalnych danych wejściowych.

3.7.1. Proces obliczania skrótu

Proces działania algorytmu *SHA-256* składa się z kilku etapów:

1. *Przygotowanie wiadomości*: do wiadomości dodawane są dodatkowe bity tak, aby jej długość była zgodna z wymaganiami algorytmu. Na końcu dołączana jest również informacja o pierwotnej długości wiadomości.
2. *Podział na bloki*: przygotowana wiadomość dzielona jest na bloki o długości 512 bitów.
3. *Inicjalizacja wartości początkowych*: algorytm wykorzystuje osiem stałych 32-bitowych wartości początkowych, które tworzą początkowy stan funkcji skrótu.
4. *Tworzenie harmonogramu wiadomości*: z każdego 512-bitowego bloku generowanych jest 64 słów o długości 32 bitów wykorzystywanych podczas dalszych obliczeń.
5. *Przetwarzanie rund*: dla każdego bloku wykonywane są 64 rundy operacji matematycznych i logicznych, takich jak: dodawanie *modulo 232*, operacje *XOR*, przesunięcia i rotacje bitowe.
6. *Aktualizacja wartości skrótu*: po zakończeniu przetwarzania danego bloku aktualizowane są wartości stanu funkcji skrótu.
7. *Wygenerowanie końcowego skrótu*: po przetworzeniu wszystkich bloków danych otrzymywany jest końcowy skrót o długości 256 bitów.

3.7.2. Zalety algorytmu SHA-256

- wysoki poziom bezpieczeństwa,
- odporność na znane ataki kryptograficzne,
- możliwość przetwarzania danych o dowolnej długości,
- szerokie zastosowanie w systemach bezpieczeństwa,
- stosunkowo wysoka wydajność działania.

3.7.3. Wady algorytmu SHA-256

- większa złożoność obliczeniowa w porównaniu z prostszymi funkcjami skrótu,
- nie służy do szyfrowania danych, a jedynie do generowania skrótów,
- dla bardzo dużych zbiorów danych może wymagać znacznych zasobów obliczeniowych.

3.7.4. Zastosowania

- weryfikacja integralności plików,
- podpisy cyfrowe,
- certyfikaty cyfrowe,
- protokół *HTTPS/TLS*,
- przechowywanie haseł (w połączeniu z dodatkowymi mechanizmami zabezpieczeń),
- technologia blockchain i kryptowaluty,
- systemy uwierzytelniania użytkowników.

Obecnie *SHA-256* jest jedną z najczęściej wykorzystywanych funkcji skrótu.

4. Technologie i narzędzia

- *Visual Studio Code* - główne środowisko programistyczne wykorzystywane do tworzenia kodu, zarządzania projektem oraz integracji z rozszerzeniami.
- *Python* - język programowania wykorzystany do implementacji aplikacji oraz mechanizmów kryptograficznych. Ze względu na prostą składnię oraz szeroki wybór bibliotek.
- *Tkinter* - standardowa biblioteka GUI języka Python. Została wykorzystana do przygotowania okien aplikacji oraz elementów umożliwiających interakcję użytkownika z programem.
- *Biblioteka Cryptography* - biblioteka kryptograficzna języka Python wykorzystana do implementacji algorytmów kryptograficznych oraz operacji związanych z zarządzaniem kluczami. Wykorzystano m.in. moduły odpowiedzialne za:

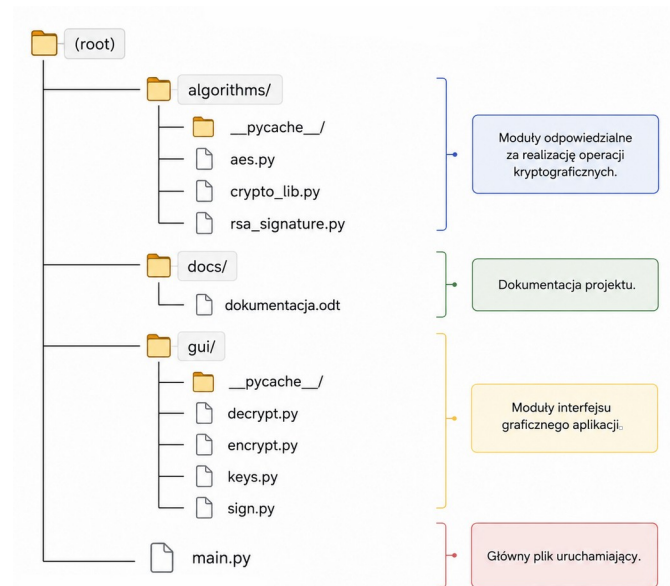
- szyfrowanie i deszyfrowanie danych przy użyciu algorytmu *AES*,
 - generowanie i obsługę kluczy *RSA*,
 - serializację oraz odczyt kluczy publicznych i prywatnych,
 - tworzenie i weryfikację podpisów cyfrowych,
 - obliczanie funkcji skrótu *SHA-256*,
 - obsługę wyjątków związanych z nieprawidłową weryfikacją podpisów cyfrowych.
- *Git* - system kontroli wersji wykorzystywany synchronizacji zmian w kodzie pomiędzy twórcami.
 - *GitHub* - platforma internetowa służąca do przechowywania repozytorium projektu oraz zarządzania kodem źródłowym. Wykorzystany do archiwizacji projektu i współpracy nad kodem.

5. Architektura aplikacji

Aplikacja została zaprojektowana w architekturze modułowej. Poszczególne funkcjonalności zostały rozdzielone na niezależne moduły odpowiedzialne za szyfrowanie, deszyfrowanie, generowanie kluczy oraz obsługę podpisów cyfrowych. Interfejs użytkownika został zaimplementowany przy użyciu biblioteki *Tkinter*, natomiast operacje kryptograficzne realizowane są przez bibliotekę *Cryptography*.

5.1. Struktura katalogów i plików

Jak wspomniano w poprzednim rozdziale (5. *Architektura aplikacji*) program został zaprojektowany w sposób modułowy. Kod źródłowy został podzielony na katalogi odpowiadające za realizację operacji kryptograficznych, obsługę interfejsu użytkownika oraz przechowywanie dokumentacji projektu. Schemat przedstawiający strukturę katalogów i plików został przedstawiony na poniższym rysunku (Rys. 1).



Rys. 1. Struktura katalogów i plików aplikacji.

Katalog „*algorithms*” zawiera moduły odpowiedzialne za implementację algorytmów kryptograficznych. Znajdują się w nim funkcje realizujące szyfrowanie i deszyfrowanie danych przy użyciu algorytmu *AES*, operacje związane z podpisem cyfrowym *RSA* oraz pomocnicze funkcje kryptograficzne.

Katalog „*gui*” zawiera moduły odpowiedzialne za obsługę graficznego interfejsu użytkownika stworzonego przy pomocy *Tkinter*. Poszczególne pliki odpowiadają za realizację konkretnych funkcji aplikacji (oddzielne ekrany aplikacji), takich jak szyfrowanie, deszyfrowanie, podpisywanie danych oraz generowanie kluczy kryptograficznych.

Katalog „*docs*” przeznaczony jest do przechowywania dokumentacji projektowej.

Głównym plikiem programu jest „*main.py*”, który odpowiada za uruchomienie aplikacji oraz inicjalizację interfejsu użytkownika. Z poziomu tego pliku następuje integracja wszystkich modułów systemu i uruchamiane są poszczególne funkcjonalności aplikacji.

5.2. Moduły systemu

5.2.1. Moduł szyfrowania

Moduł szyfrowania odpowiada za zabezpieczanie danych przy użyciu wybranego algorytmu. Użytkownik może wskazać plik wejściowy, lokalizację pliku wynikowego oraz wybrać parametry szyfrowania. Moduł może automatycznie generować klucz szyfrowania oraz inne wartości konieczne. Dodatkowo istnieje możliwość zaszyfrowania klucza *AES* przy użyciu klucza publicznego *RSA* odbiorcy, co pozwala na wykorzystanie szyfrowania hybrydowego.

Interfejs modułu szyfrowania został zaimplementowany w pliku „*encrypt.py*” jako klasa *EncryptFrame* (dziedzicząca z klasy bazowej *tk.Frame*). Odpowiada ona za utworzenie elementów graficznych takich jak przyciski wyboru wspomnianych we wcześniejszym akapicie opcji. Struktura ekranu modułu *Encrypt* przedstawiona została na poniższym rysunku (Rys. 2.)

Rys. 2. Okno modułu szyfrowania.

Jako „Plik wejściowy” użytkownik może wskazać dowolny plik *.docx*, *.txt* czy *.mp4* – aplikacja nie nakłada ograniczeń na typ szyfrowanych danych. Jako „Plik wyjściowy” użytkownik wskazuje lokalizację oraz nazwę pliku wynikowego (bez rozszerzenia). W lokalizacji tej zapisywane są również pliki klucza szyfrującego, „Nonce/IV” (jeżeli wykorzystywane) oraz zaszyfrowany klucz *AES* (jeżeli wybrano opcję szyfrowania hybrydowego).

Po kliknięciu przycisku „Szyfruj” uruchamiana jest metoda *encrypt()* (Listing 1.), która w pierwszej kolejności sprawdza czy użytkownik wskazał plik wejściowy oraz plik wyjściowy. Jeżeli wybrano opcję szyfrowania klucza *AES* za pomocą *RSA*, program sprawdza również, czy został podany plik klucza publicznego *RSA*.

W przypadku wykrycia błędów (np.: brak plików, niepowodzenie szyfrowania) w polu tekstowym (*status*) obok przycisku szyfruj wyświetlany jest stosowny komunikat. Pole to służy do komunikacji statusu szyfrowania do użytkownika i jako prosty „*debugger*” informujący o występujących błędach.

Właściwe szyfrowanie realizowane jest przez funkcję *encrypt_dispatcher()* (Listing 2.) znajdującą się w module „*crypto_lib.py*”. Funkcja ta sprawdza wybrany algorytm, a następnie generuje losowy klucz *AES* o długości 32 bajtów (*AES-256*) oraz wartość *nonce* o długości 12

bajtów. Dane z pliku wejściowego są odczytywane są binarnie i szyfrowane przy użyciu klas *AES* z biblioteki *cryptography*.

Wynikiem działania modułu jest zestaw plików zapisanych w postaci binarnej:

- plik główny (nazwa wskazana przez użytkownika) – zawierający zaszyfrowane dane,
- plik *.nonce* – zawierający wartości *Nonce/IV* wykorzystywane przez algorytm szyfrowania,
- plik *.key* - zawierający klucz AES,
- plik *.key.enc* – alternatywa dla pliku *.key*, tworzony jest w przypadku wybrania opcji szyfrowania hybrydowego.

Listing 1. Kod metody *encrypt()* modułu szyfrującego.

```
def encrypt(self):

    if not self.input_file:
        messagebox.showerror( "Błąd", "Nie wybrano pliku wejściowego")
        return

    if not self.output_file:
        messagebox.showerror("Błąd", "Nie wybrano pliku wyjściowego")
        return

    rsa_enabled = self.rsa_encrypt_var.get()

    if rsa_enabled and not self.rsa_key_file_public:
        messagebox.showerror("Błąd", "Nie wybrano klucza publicznego RSA")
        return

    try:
        self.busy_status_label.config(text="Szyfrowanie...")

        selected_algorithm = self.selected_aes_mode.get()

        encrypt_dispatcher(
            algorithm=selected_algorithm,
            input_file=self.input_file,
            output_file=self.output_file,
            rsa_enabled=rsa_enabled,
            rsa_public_key_file=self.rsa_key_file_public
        )

        self.busy_status_label.config(text="Szyfrowanie zakończone")

        messagebox.showinfo("Sukces", "Plik został zaszyfrowany")

    except Exception as e:
        self.busy_status_label.config(text="Błąd")

        messagebox.showerror("Błąd szyfrowania",str(e))
```

Listing 2. Kod metody *encrypt_dispatcher()*.

```
def encrypt_dispatcher( algorithm, input_file, output_file, rsa_enabled=False, rsa_public_key_file=None ):
    if algorithm == "AES-256-GCM":
        aes_key = generate_aes_key()
        nonce = generate_nonce()
        encrypt_file_aes_gcm( input_file=input_file, output_file=output_file, aes_key=aes_key, nonce=nonce )
        base_name = output_file

        key_file = base_name + ".key"
        nonce_file = base_name + ".nonce"

    if rsa_enabled:
```



```

        encrypted_key = encrypt_aes_key_with_rsa( aes_key, rsa_public_key_file )
        save_binary_file(key_file + ".enc", encrypted_key )
    else:
        save_binary_file( key_file, aes_key )

    save_binary_file( nonce_file, nonce )
)
else:
    raise EncryptionError( f"Nieobsługiwany algorytm: {algorithm}" )

```

Listing 3. Metody wykorzystywane przez `encrypt_dispatcher()`.

```

def generate_aes_key():
    return os.urandom(32)

def generate_nonce():
    return os.urandom(12)

def save_binary_file(path, data):
    with open(path, "wb") as f:
        f.write(data)

def encrypt_file_aes_gcm( input_file, output_file, aes_key, nonce ):
    try:
        with open(input_file, "rb") as f:
            plaintext = f.read()

        aesgcm = AESGCM(aes_key)

        ciphertext = aesgcm.encrypt( nonce, plaintext, None)

        with open(output_file, "wb") as f:
            f.write(ciphertext)

    except Exception as e:
        raise EncryptionError(f"Błąd szyfrowania AES-GCM: {str(e)}")

def encrypt_aes_key_with_rsa( aes_key, public_key_file ):
    try:
        with open(public_key_file, "rb") as key_file:
            public_key = load_pem_public_key( key_file.read() )

        encrypted_key = public_key.encrypt( aes_key,
            padding.OAEP(
                mgf=padding.MGF1(
                    algorithm=hashes.SHA256()
                ),
                algorithm=hashes.SHA256(),
                label=None
            )
        )

        return encrypted_key

    except Exception as e:
        raise EncryptionError(
            f"Błąd szyfrowania klucza AES kluczem RSA: {str(e)}"
        )

```

5.2.2. Moduł deszyfrowania

Moduł deszyfrowania odpowiada za odzyskiwanie danych, które zostały wcześniej zabezpieczone przy użyciu modułu szyfrowania.

Interfejs modułu deszyfrowania został zaimplementowany w pliku „*decrypt.py*” jako klasa *DecryptFrame*, dziedzicząca z klasy bazowej *tk.Frame*. Odpowiada ona za utworzenie elementów graficznych widocznych w interfejsie użytkownika. Struktura ekranu modułu *Decrypt* przedstawiona została na poniższym rysunku (Rys. 3).

The image shows a graphical user interface for a decryption module. At the top, there is a horizontal bar with four buttons: 'Encrypt', 'Decrypt', 'Sign', and 'Keys'. Below this bar, the interface is organized into several sections. The first section contains two labels, 'Plik wejściowy:' and 'Plik wyjściowy:', each followed by a 'Przeglądaj' (Browse) button. The second section has a label 'Tryb AES:' followed by a dropdown menu currently showing 'AES-256-GCM'. Below this is a label 'Nonce/IV:' followed by another 'Przeglądaj' button. The third section contains a label 'Klucz AES:' followed by a 'Przeglądaj' button. Below this is a checkbox labeled 'Klucz AES zaszyfrowany kluczem RSA'. The fourth section has a label 'Klucz prywatny RSA odbiorcy:' followed by a 'Przeglądaj' button. At the bottom of the window, there is a large 'Odszyfruj' (Decrypt) button and the text 'Not executed yet'.

Rys. 3. Okno modułu deszyfrowania.

W celu poprawnego odszyfrowania użytkownik musi zadeklarować:

- *Plik wejściowy* – plik zawierający zaszyfrowane dane w formacie binarnym,
- *Plik wyjściowy* – lokalizację oraz nazwę pliku, do którego zapisane zostaną dane po odszyfrowaniu,
- *Nonce/IV* - plik *.nonce*,
- plik *.key* lub plik *.key.enc* – plik zawierający klucz AES lub klucz AES zaszyfrowany kluczem publicznym RSA,
- plik klucza prywatnego – do rozszyfrowania klucza AES (jeżeli zaszyfrowany).

Po kliknięciu przycisku „Odszyfruj” uruchamiana jest metoda *decrypt()* (Listing 4.), która sprawdza, czy użytkownik wskazał wszystkie wspomniane wcześniej pliki. Brak któregośkolwiek z wymaganych elementów, podobnie jak w przypadku modułu *Encrypt*, powoduje wyświetlenie komunikatu błędu.

Właściwe odszyfrowanie realizowane jest przez funkcję *decrypt_dispatcher()* (Listing 5.) znajdującą się w module „*crypto_lib.py*”. Funkcja ta sprawdza wybrany tryb szyfrowania, pobiera klucz AES i wartość Nonce/IV. Jeżeli nie użyto szyfrowania hybrydowego, klucz AES odczytywany jest bezpośrednio z pliku *.key*. W przeciwnym wypadku funkcja korzysta z

decrypt_aes_key_with_rsa() (Listing 6.), która odszyfrowuje plik *.key.enc* przy użyciu klucza prywatnego RSA odbiorcy.

Po pobraniu klucza AES oraz wartości *nonce* wywoływana jest funkcja *decrypt_file_aes_gcm()*. Funkcja ta odczytuje zaszyfrowany plik w postaci binarnej, tworzy obiekt klasy *AESGCM*, a następnie wykonuje operację odszyfrowania. Otrzymane dane jawne zapisywane są do wskazanego przez użytkownika pliku wyjściowego. Wynikiem działania modułu jest plik zawierający odszyfrowane dane w pierwotnej postaci.

Moduł deszyfrowania posiada ograniczenie związane z koniecznością znajomości oryginalnego rozszerzenia szyfrowanego pliku. Aplikacja nie zapisuje informacji o typie pliku przy szyfrowaniu i nie narzuca rozszerzenia pliku wynikowego podczas odszyfrowywania. W związku z tym użytkownik musi samodzielnie nadać plikowi wyjściowemu rozszerzenie odpowiadające jego pierwotnemu formatowi. W przypadku braku tej wiedzy odczytanie/interpretacja zawartości odszyfrowanego pliku może być utrudnione lub niemożliwe.

Listing 4. Kod metody *decrypt()*.

```
def decrypt(self):
    if not self.input_file:
        messagebox.showerror("Błąd", "Brak pliku wejściowego")
        return
    if not self.output_file:
        messagebox.showerror("Błąd", "Brak pliku wyjściowego")
        return
    if not self.aes_key_file:
        messagebox.showerror("Błąd", "Brak klucza AES")
        return
    rsa_enabled = self.aes_key_var.get()
    if rsa_enabled and not self.rsa_key_file_private:
        messagebox.showerror("Błąd", "Brak klucza prywatnego RSA")
        return

    try:
        self.busy_status_label.config(text="Odszyfrowywanie...")
        algorithm = self.selected_aes_mode.get()
        decrypt_dispatcher(
            algorithm=algorithm,
            input_file=self.input_file,
            output_file=self.output_file,
            aes_key_file=self.aes_key_file,
            rsa_enabled=rsa_enabled,
            rsa_private_key_file=self.rsa_key_file_private,
            nonce_file=self.nonce_iv_file)

        self.busy_status_label.config(text="Gotowe")
        messagebox.showinfo("OK", "Plik odszyfrowany")

    except Exception as e:
        self.busy_status_label.config(text="Błąd")
        messagebox.showerror("Błąd odszyfrowania", str(e))
```

Listing 5. Kod metody *decrypt_dispatcher()*.

```
def decrypt_dispatcher(algorithm, input_file, output_file, aes_key_file=None, nonce_file=None, rsa_enabled=False,
    rsa_private_key_file=None ):

    if algorithm != "AES-256-GCM":
        raise ValueError("Nieobsługiwany tryb AES")

    if rsa_enabled:
        aes_key = decrypt_aes_key_with_rsa( encrypted_key_path=aes_key_file, private_key_path=rsa_private_key_file
    )
```

```

else:
    with open(aes_key_file, "rb") as f:
        aes_key = f.read()

if not nonce_file:
    raise ValueError("Brak pliku nonce/IV")

with open(nonce_file, "rb") as f:
    nonce = f.read()

decrypt_file_aes_gcm(
    input_file=input_file,
    output_file=output_file,
    aes_key=aes_key,
    nonce=nonce )

```

Listing 6. Metody wykorzystywane przez `decrypt_dispatcher()`.

```

def decrypt_aes_key_with_rsa(encrypted_key_path, private_key_path):
    with open(private_key_path, "rb") as f: private_key = load_pem_private_key(f.read(), password=None)
    with open(encrypted_key_path, "rb") as f:
        encrypted_key = f.read()

    aes_key = private_key.decrypt( encrypted_key,
        padding.OAEP( mgf=padding.MGF1(algorithm=hashes.SHA256()), algorithm=hashes.SHA256(), label=None ) )
    return aes_key

def decrypt_file_aes_gcm(input_file, output_file, aes_key, nonce):
    try:
        with open(input_file, "rb") as f:
            ciphertext = f.read()
        aesgcm = AESGCM(aes_key)
        plaintext = aesgcm.decrypt(nonce, ciphertext, None )

        with open(output_file, "wb") as f:
            f.write(plaintext)
    except Exception as e:
        raise EncryptionError(f"Błąd odszyfrowania AES-GCM: {str(e)}")

```

5.2.3. Moduł podpisu cyfrowego

Moduł podpisu cyfrowego odpowiada za generowanie oraz weryfikację podpisów z wykorzystaniem algorytmu *RSA* oraz funkcji *SHA-256*.

Interfejs modułu został zaimplementowany w pliku „*sign.py*” jako klasa *SignFrame*, dziedzicząca z klasy bazowej *tk.Frame*. Odpowiada ona za utworzenie elementów graficznych widocznych w zakładce *Sign*. Struktura ekranu modułu przedstawiona została na poniższym rysunku.

Rys. 4. Okno modułu podpisu cyfrowego i weryfikacji podpisu.

W części odpowiedzialnej za podpisywanie użytkownik wskazuje plik przeznaczony do podpisania, klucz prywatny *RSA* nadawcy oraz lokalizację pliku podpisu. Aplikacja umożliwia podpisywanie dowolnych typów plików. Wygenerowany podpis zapisywany jest jako osobny plik binarny.

Po kliknięciu przycisku Podpisz uruchamiana jest metoda *sign()* (Listing 7.), która sprawdza poprawność wprowadzonych danych (obecność pliku przeznaczonego do podpisania, klucza prywatnego *RSA* oraz lokalizacji pliku podpisu). W przypadku braku wymaganych danych wyświetlany jest odpowiedni komunikat w polu statusu.

Proces podpisywania realizowany jest przez funkcję *sign_file()* (Listing 8.) znajdującą się w module „*rsa_signature.py*”. Funkcja odczytuje klucz prywatny *RSA* zapisany w formacie *PEM*, a

następnie pobiera zawartość wskazanego pliku. Kolejnym krokiem jest wygenerowanie podpisu cyfrowego przy użyciu metody `private_key.sign()` (Listing 8.). W procesie tym wykorzystywany jest algorytm RSA wraz z mechanizmem dopełniania PSS (*Probabilistic Signature Scheme*) oraz funkcją skrótu SHA-256.

Druga część modułu odpowiada za weryfikację podpisu cyfrowego. W tym celu poprawnej weryfikacji podpisu użytkownik wskazuje plik do weryfikacji, klucz publiczny RSA nadawcy oraz plik zawierający podpis cyfrowy. Po wybraniu przycisku „Zweryfikuj” uruchamiana jest metoda `verify_signature()` (Listing 7.), która sprawdza kompletność wymaganych danych, a następnie wywołuje funkcję `verify_signature()` (Listing 8.) znajdującą się w module „rsa_signature.py”.

Proces weryfikacji polega na odczytaniu klucza publicznego RSA, danych z pliku oraz podpisu cyfrowego. Następnie wykonywana jest operacja `public_key.verify()`, wykorzystująca ten sam schemat RSA-PSS oraz funkcję SHA-256, które zostały użyte podczas podpisywania. Jeżeli podpis jest poprawny użytkownik otrzymuje komunikat „Podpis poprawny”. W przeciwnym przypadku zgłaszany jest wyjątek, a aplikacja informuje użytkownika o niepoprawnym podpisie.

Zastosowane rozwiązanie pozwala wykryć każdą modyfikację podpisanego pliku (niewielka zmiana zawartości powoduje wygenerowanie innego skrótu). Dzięki temu moduł zapewnia integralność danych oraz umożliwia potwierdzenie tożsamości nadawcy.

Listing 7. Metody `sign()` i `verify_signature()`.

```
def sign(self):
    try:
        if not self.to_sign_file:
            self.busy_status_label_1.config(text="Wybierz plik")
            return
        if not self.private_rsa_key_file:
            self.busy_status_label_1.config(text="Wybierz klucz prywatny")
            return
        if not self.output_file:
            self.busy_status_label_1.config(text="Wybierz plik podpisu")
            return
        sign_file(self.to_sign_file, self.private_rsa_key_file, self.output_file)
        self.busy_status_label_1.config(text="Plik podpisany")

    except Exception as e:
        self.busy_status_label_1.config(text=f"Błąd: {e}")

def verify_signature(self):
    try:
        if not self.to_verify_file:
            self.busy_status_label_2.config(text="Wybierz plik")
            return
        if not self.rsa_key_file_public:
            self.busy_status_label_2.config(text="Wybierz klucz publiczny")
            return
        if not self.signature_file:
            self.busy_status_label_2.config(text="Wybierz podpis")
            return
        result = verify_signature(self.to_verify_file, self.rsa_key_file_public, self.signature_file)

        if result:
            self.busy_status_label_2.config(text="Podpis poprawny")
        else:
            self.busy_status_label_2.config(text="Podpis NIEPOPRAWNY")

    except Exception as e:
        self.busy_status_label_2.config(text=f"Błąd: {e}")
```

Listing 7. Metody `verify_signature()` i `sign_file()`.

```
def sign_file(file_path, private_key_path, output_signature_path):
    with open(private_key_path, "rb") as f:
        private_key = serialization.load_pem_private_key( f.read(), password=None )

    with open(file_path, "rb") as f:
        data = f.read()

    signature = private_key.sign(
        data,
        padding.PSS( mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH ), hashes.SHA256() )

    with open(output_signature_path, "wb") as f:
        f.write(signature)

def verify_signature(file_path, public_key_path, signature_path):
    with open(public_key_path, "rb") as f:
        public_key = serialization.load_pem_public_key(f.read())

    with open(file_path, "rb") as f:
        data = f.read()

    with open(signature_path, "rb") as f:
        signature = f.read()

    try:
        public_key.verify( signature, data, padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH ), hashes.SHA256() )
        return True
    except InvalidSignature:
        return False
```

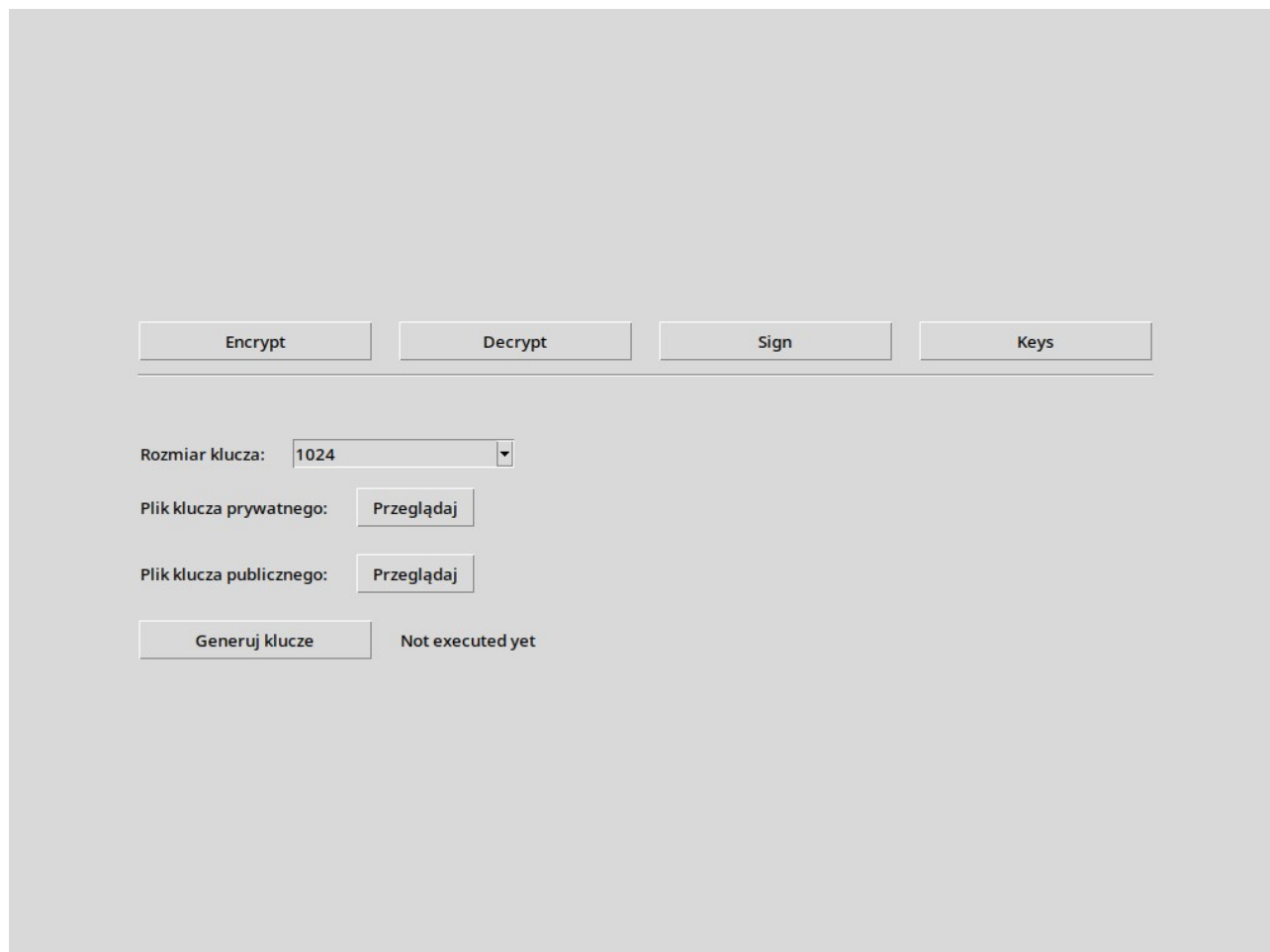
5.2.5. Moduł generowania kluczy

Moduł generowania kluczy odpowiada za tworzenie par kluczy *RSA*, które są następnie wykorzystywane w pozostałych częściach aplikacji. Klucz publiczny może być używany do szyfrowania klucza *AES* oraz do weryfikacji podpisu cyfrowego. Klucz prywatny służy natomiast do odszyfrowywania klucza *AES* oraz tworzenia podpisów cyfrowych.

Interfejs modułu generowania kluczy został zaimplementowany w pliku „*keys.py*” jako klasa *KeysFrame*, dziedzicząca z klasy bazowej *tk.Frame*. Odpowiada ona za utworzenie elementów graficznych widocznych w zakładce *Keys*, takich jak lista wyboru rozmiaru klucza, przyciski wskazania lokalizacji oraz przyciski uruchamiający proces generowania kluczy. Struktura ekranu modułu przedstawiona została na poniższym rysunku (*Rys. 5*).

Użytkownik wybiera rozmiar klucza *RSA* z dostępnej listy, wskazuje lokalizację zapisu pliku klucza prywatnego oraz pliku klucza publicznego. W prezentowanym module dostępny jest m.in. 1024 i 2048 bitowy rozmiar klucza, jednak konstrukcja interfejsu pozwala na rozszerzenie listy o kolejne wartości.

Po kliknięciu przycisku „*Generuj klucze*” uruchamiana jest metoda *generate_keys()* (*Listing 8.*), która w sprawdza, czy użytkownik wskazał lokalizacje obu plików wyjściowych. Jeżeli nie wybrano pliku klucza prywatnego lub publicznego, w polu statusu wyświetlany jest stosowny komunikat.



Rys. 5. Okno modułu zarządzania kluczami.

Właściwe generowanie kluczy realizowane jest przez funkcję *generate_keys()* (Listing 9.) znajdującą się w module „rsa_signature.py”. Funkcja ta wykorzystuje metodę *rsa.generate_private_key()* z biblioteki *cryptography*, przyjmując wykładnik publiczny 65537 oraz rozmiar klucza wybrany przez użytkownika. Kolejno na podstawie wygenerowanego klucza prywatnego tworzony jest odpowiadający mu klucz publiczny.

Wynikiem działania modułu są dwa pliki:

- plik klucza prywatnego RSA – zapisany jest w formacie *PKCS8*,
- plik klucza publicznego RSA – zapisany w formacie *SubjectPublicKeyInfo*.

Listing 8. Metoda *generate_keys()*.

```
def generate_keys(self):
    try:
        if not self.output_private_key_file or not self.output_public_key_file:
            self.busy_status_label.config(text="Wybierz pliki wyjściowe")
            return
        key_size = int(self.selected_key_size.get())
        generate_keys(self.output_private_key_file, self.output_public_key_file, key_size)

        self.busy_status_label.config(text="Klucze wygenerowane")

    except Exception as e:
        self.busy_status_label.config(text=f"Błąd: {e}")
```


Listing 9. Metoda `generate_keys()` modułu `rsa_signature.py`.

```
def generate_keys(private_key_path, public_key_path, key_size=2048):
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=key_size
    )
    public_key = private_key.public_key()

    with open(private_key_path, "wb") as f:
        f.write(
            private_key.private_bytes(
                encoding=serialization.Encoding.PEM,
                format=serialization.PrivateFormat.PKCS8,
                encryption_algorithm=serialization.NoEncryption()
            )
        )

    with open(public_key_path, "wb") as f:
        f.write(
            public_key.public_bytes(
                encoding=serialization.Encoding.PEM,
                format=serialization.PublicFormat.SubjectPublicKeyInfo
            )
        )
```

6. Bezpieczeństwo systemu i możliwe zagrożenia

Bezpieczeństwo zaprojektowanego systemu/programu opiera się na zastosowaniu mechanizmów kryptograficznych, których zadaniem jest ochrona danych przed nieautoryzowanym dostępem oraz modyfikacją. W celu osiągnięcia tego zadania wykorzystane zostało połączenie kryptografii symetrycznej, asymetrycznej oraz funkcji skrótu.

Należy jednak zaznaczyć, że samo zastosowanie silnych algorytmów kryptograficznych nie jest w stanie zapewnić pełnego bezpieczeństwa systemu. Równie ważne są sposoby przechowywania kluczy, odpowiednie procedury organizacyjne oraz w głównej mierze świadomość użytkownika.

W związku z powyższym konieczne jest rozpatrywanie kwestii bezpieczeństwa proponowanego rozwiązania pod kątem zarówno technicznym jak i użytkowym, czy organizacyjnym.

6.1. Założenia bezpieczeństwa systemu

System zakłada, że użytkownik korzysta z aplikacji w bezpiecznym środowisku, czyli na urządzeniu wolnym od złośliwego oprogramowania i niedostępnym dla osób nieupoważnionych. Zakłada się również, że użytkownik świadomie zarządza wygenerowanymi przez siebie kluczami i nie udostępnia ich osobom nieuprawnionym.

Podstawowe założenia bezpieczeństwa systemu są następujące:

- klucz prywatny *RSA* powinien być znany wyłącznie jego właścicielowi,
- klucz publiczny może być udostępniany innym użytkownikom,
- pliki zawierające klucze powinny być przechowywane w bezpiecznej lokalizacji,

- zaszyfrowane dane mogą być przesyłane kanałami mniej zaufanymi, pod warunkiem że klucz AES został odpowiednio zabezpieczony,
- podpis cyfrowy powinien być stosowany wtedy, gdy odbiorca musi mieć pewność, że plik pochodzi od właściwego nadawcy i nie został zmodyfikowany.

6.2. Ochrona poufności danych

Poufność danych w systemie polega na uniemożliwieniu odczytania zawartości pliku przez osoby postronne. Program zabezpiecza dane poprzez przekształcenie ich do postaci zaszyfrowanej, przy pomocy zastosowanych algorytmów, która bez odpowiedniego klucza jest nieczytelna.

Takie rozwiązanie pozwala bezpieczniej przechowywać pliki na komputerze, nośniku zewnętrznym lub przysyłać je przez mniej zaufane kanały komunikacji. Nawet w przypadku przechwycenia zaszyfrowanego pliku osoba nieuprawniona nie powinna uzyskać dostępu do jego treści bez posiadania właściwego klucza.

6.3. Bezpieczna wymiana kluczy

Jednym z ważniejszych, jak nie najważniejszym, elementem bezpieczeństwa systemu jest sposób przekazywania danych pomiędzy użytkownikami. Samo zaszyfrowanie pliku zwiększa bezpieczeństwo, jednak konieczne jest również odpowiednie zabezpieczenie klucza potrzebnego do jego odszyfrowania.

Program umożliwia zabezpieczenie klucza szyfrującego w taki sposób, aby mógł go odczytać wyłącznie właściwy odbiorca. Dzięki temu zaszyfrowany plik oraz zabezpieczony klucz mogą zostać przekazane odbiorcy bez ujawniania treści.

6.4. Integralność i autentyczność danych

System może również kontrolę integralności i autentyczności plików. Integralność oznacza możliwość sprawdzenia, czy plik nie został zmodyfikowany po jego utworzeniu lub podpisaniu. Autentyczność pozwala natomiast upewnić się, że plik pochodzi od właściwego nadawcy.

W praktyce mechanizm podpisu cyfrowego może być wykorzystany w sytuacjach, w których odbiorca chce mieć pewność, że dokument nie został zmieniony podczas przesyłania. Dotyczy to na przykład umów, raportów oraz dokumentów firmowych, których ew. zmiana mogłaby wpłynąć np.: na finanse organizacji.

Jeżeli podpis nie przejdzie poprawnej weryfikacji, użytkownik powinien uznać plik za potencjalnie zmieniony lub pochodzący z niepewnego źródła i zgłosić to do odpowiedniego działu.

6.5. Możliwe zagrożenia

Pomimo zastosowania mechanizmów kryptograficznych system nie jest całkowicie odporny na zagrożenia. Możliwe naruszenia mogą wynikać z błędów użytkownika, niewłaściwego przechowywania plików lub zagrożeń występujących poza samą aplikacją.

Do najważniejszych zagrożeń należą:

- przejęcie klucza prywatnego przez osobę nieuprawnioną,
- zapisanie kluczy w niezabezpieczonej lokalizacji,
- przesyłanie klucza prywatnego innym osobom,
- utrata pliku klucza lub pliku potrzebnego do odszyfrowania danych,
- zainfekowanie komputera złośliwym oprogramowaniem,
- używanie programu na niezaufanym urządzeniu,
- błędne wskazanie plików podczas szyfrowania lub odszyfrowywania,
- ataki socjotechniczne prowadzące do ujawnienia kluczy lub poufnych danych.

Szczególnie niebezpieczne jest przejęcie klucza prywatnego, ponieważ może ono umożliwić nieuprawnione odszyfrowywanie danych lub podszywanie się pod właściciela klucza podczas tworzenia podpisów cyfrowych.

System nie chroni również przed sytuacją, w której osoba atakująca ma dostęp do danych przed ich zaszyfrowaniem lub po ich odszyfrowaniu. Dlatego (jak podano w rozdziale 7.2) ważne jest, aby korzystać z programu wyłącznie na urządzeniach, które są odpowiednio zabezpieczone.

6.7. Metody ograniczania ryzyka

W celu zapewnienia jak największego poziomu bezpieczeństwa systemu należy stosować odpowiednie praktyki użytkowe. Najważniejszą zasadą jest ochrona klucza prywatnego oraz niedopuszczanie do sytuacji, w której osoba nieuprawniona uzyska dostęp do plików umożliwiających odszyfrowanie danych wrażliwych.

W celu ograniczenia ryzyka zaleca się:

- przechowywanie kluczy prywatnych w bezpiecznej lokalizacji,
- niewysyłanie kluczy prywatnych przez pocztę elektroniczną lub komunikatory,
- stosowanie kopii zapasowych kluczy,
- oddzielne przechowywanie zaszyfrowanych plików i kluczy,
- regularne aktualizowanie systemu operacyjnego i oprogramowania,
- korzystanie z programu wyłącznie na zaufanych urządzeniach,
- sprawdzanie poprawności podpisu cyfrowego przy ważnych dokumentach,
- zachowanie ostrożności przy odbieraniu plików z nieznanych źródeł.

Innymi słowy, kluczowym będzie odpowiednie przeszkolenie personelu wykorzystującego oprogramowanie w zakresie bezpiecznego podstaw cyberbezpieczeństwa, rozpoznawania potencjalnych zagrożeń oraz prawidłowego przekazywania plików wrażliwych. Użytkownicy

powinni wiedzieć, które elementy mogą być udostępniane innym osobom, a które muszą pozostać poufne, na przykład klucz prywatny.

Istotne jest również określenie procedur postępowania w sytuacjach awaryjnych, takich jak utrata klucza prywatnego, podejrzenie jego przejęcia lub przypadkowe usunięcie plików potrzebnych do odszyfrowania danych.

7. Możliwości wdrożenia i zastosowania systemu

System, ze względu na prostą budowę i niepełne uwzględnienie wygody i łatwości wykorzystania nie powinien być traktowany jako kompletne rozwiązanie, lecz jako aplikacja demonstrująca praktyczne zastosowanie wybranych mechanizmów kryptograficznych. W przypadku wdrożenia w organizacji konieczne byłoby uzupełnienie go o dodatkowe procedury, takie jak polityka zarządzania kluczami, kopie zapasowe czy kontrola dostępu.

Ze względu na prostą budowę aplikacja może być przydatna w celach edukacyjnych, testowych lub jako wsparcie przy zabezpieczaniu pojedynczych plików. Może również posłużyć jako punkt wyjścia do dalszego rozwoju bardziej rozbudowanego systemu ochrony danych.

7.1. Zastosowanie systemu i przykładowa wymiana plików

System mógłby zostać wykorzystany w niewielkiej organizacji jako narzędzie wspierające bezpieczną wymianę pojedynczych dokumentów pomiędzy pracownikami.

W praktycznym zastosowaniu użytkownik mógłby zaszyfrować plik przed wysłaniem go pocztą elektroniczną.

Przykładowa wymiana pliku w organizacji mogłaby przebiegać w następujący sposób:

1. Odbiorca udostępnia nadawcy swój klucz publiczny.
2. Nadawca wybiera dokument, który chce zabezpieczyć.
3. Program szyfruje plik oraz zabezpiecza klucz potrzebny do jego odszyfrowania.
4. Nadawca opcjonalnie podpisuje dokument swoim kluczem prywatnym.
5. Nadawca przesyła odbiorcy zaszyfrowany plik, zabezpieczony klucz oraz ewentualny podpis.
6. Odbiorca odszyfrowuje plik przy użyciu swojego klucza prywatnego.
7. Odbiorca może dodatkowo zweryfikować podpis cyfrowy nadawcy.

Obecna wersja programu najlepiej sprawdziłaby się jako narzędzie do szyfrowania plików przed przechowaniem ich w chmurze.

7.2. Ograniczenia obecnej wersji systemu

Obecna wersja programu wymaga od użytkownika ręcznego wskazywania plików, kluczy oraz lokalizacji zapisu wyników. Może to prowadzić do pomyłek, szczególnie w przypadku osób, które nie mają doświadczenia z obsługą plików kluczy lub szeroko pojętego sprzętu elektronicznego.

Ograniczeniem jest również brak centralnego katalogu użytkowników i kluczy publicznych. W praktycznym zastosowaniu organizacja musiałaby samodzielnie ustalić, w jaki sposób użytkownicy przekazują sobie klucze publiczne oraz jak upewniają się, że korzystają z właściwego klucza odbiorcy.

7.3. Możliwości dalszego rozwoju

W przyszłości system mógłby zostać rozbudowany o funkcje zwiększające wygodę i bezpieczeństwo użytkowania. Przydatne byłoby między innymi automatyczne pakowanie zaszyfrowanego pliku, zabezpieczonego klucza oraz plików pomocniczych do jednego archiwum.

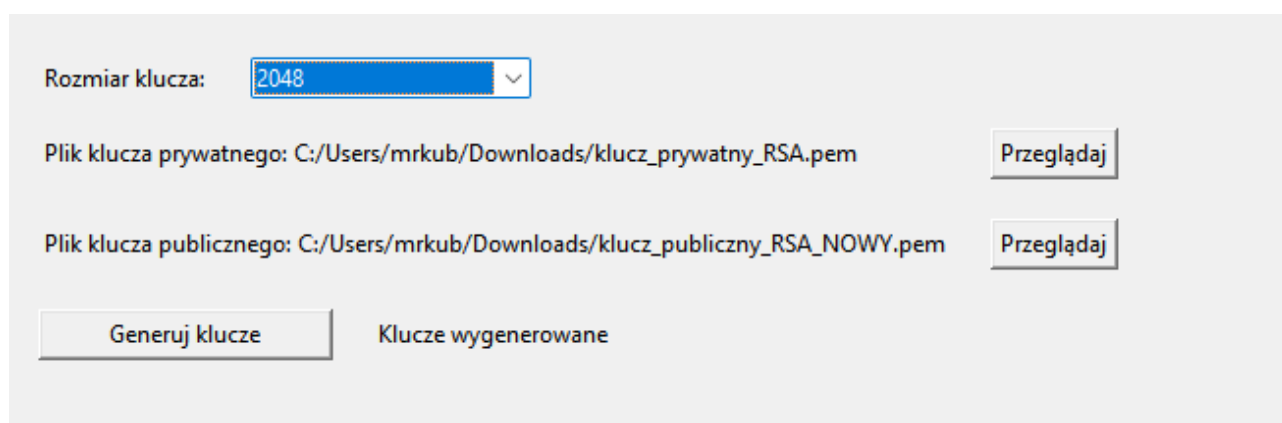
Uspewnieniem byłoby dodanie katalogu użytkowników i kluczy publicznych, dzięki któremu nadawca mógłby wybrać odbiorcę z listy, a program automatycznie używałby właściwego klucza. Zmniejszyłoby to ryzyko pomyłki podczas szyfrowania pliku.

Oprogramowanie mógłby zostać również rozbudowane o bardziej przyjazny interfejs prowadzący użytkownika krok po kroku przez proces szyfrowania, podpisywania i odszyfrowywania danych, który ograniczał by możliwe wystąpienia błędów spowodowanych elementem ludzkim.

8. Przykłady użycia i testowanie systemu

8.1. Generowanie kluczy RSA

Pierwszym etapem testowania systemu było sprawdzenie poprawności działania modułu generowania kluczy RSA. Użytkownik wybrał rozmiar klucza 2048 bitów oraz wskazał lokalizację zapisania klucza publicznego i prywatnego.



Rys. 6: Wygenerowanie kluczy prywatnych i publicznych.

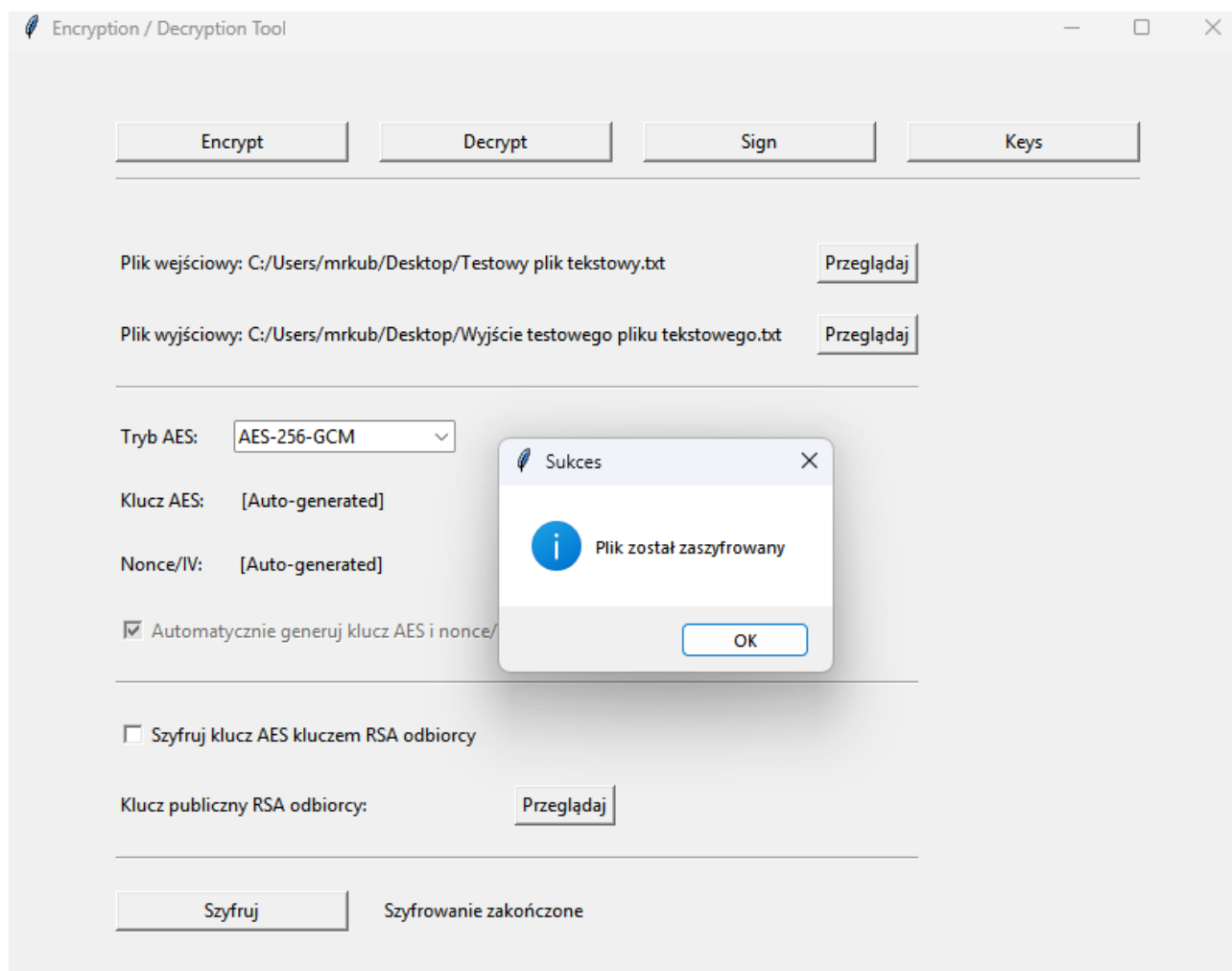
Po uruchomieniu procesu aplikacja wygenerowała parę kluczy kryptograficznych:

- klucz prywatny RSA zapisany w formacie *PEM* (*PKCS8*),
- klucz publiczny RSA zapisany w formacie *PEM* (*SubjectPublicKeyInfo*).

Wygenerowane klucze zostały następnie wykorzystane podczas kolejnych testów obejmujących szyfrowanie hybrydowe oraz podpis cyfrowy. Program poprawnie odczytał oba pliki, co potwierdziło prawidłowość procesu generowania oraz zapisu kluczy.

8.2. Szyfrowanie i odszyfrowanie dokumentu

W ramach testów przeprowadzono proces szyfrowania oraz odszyfrowania dokumentu tekstowego. Użytkownik wskazał plik wejściowy, wybrał lokalizację pliku wynikowego, a następnie uruchomił proces szyfrowania przy użyciu algorytmu *AES-256-GCM*.

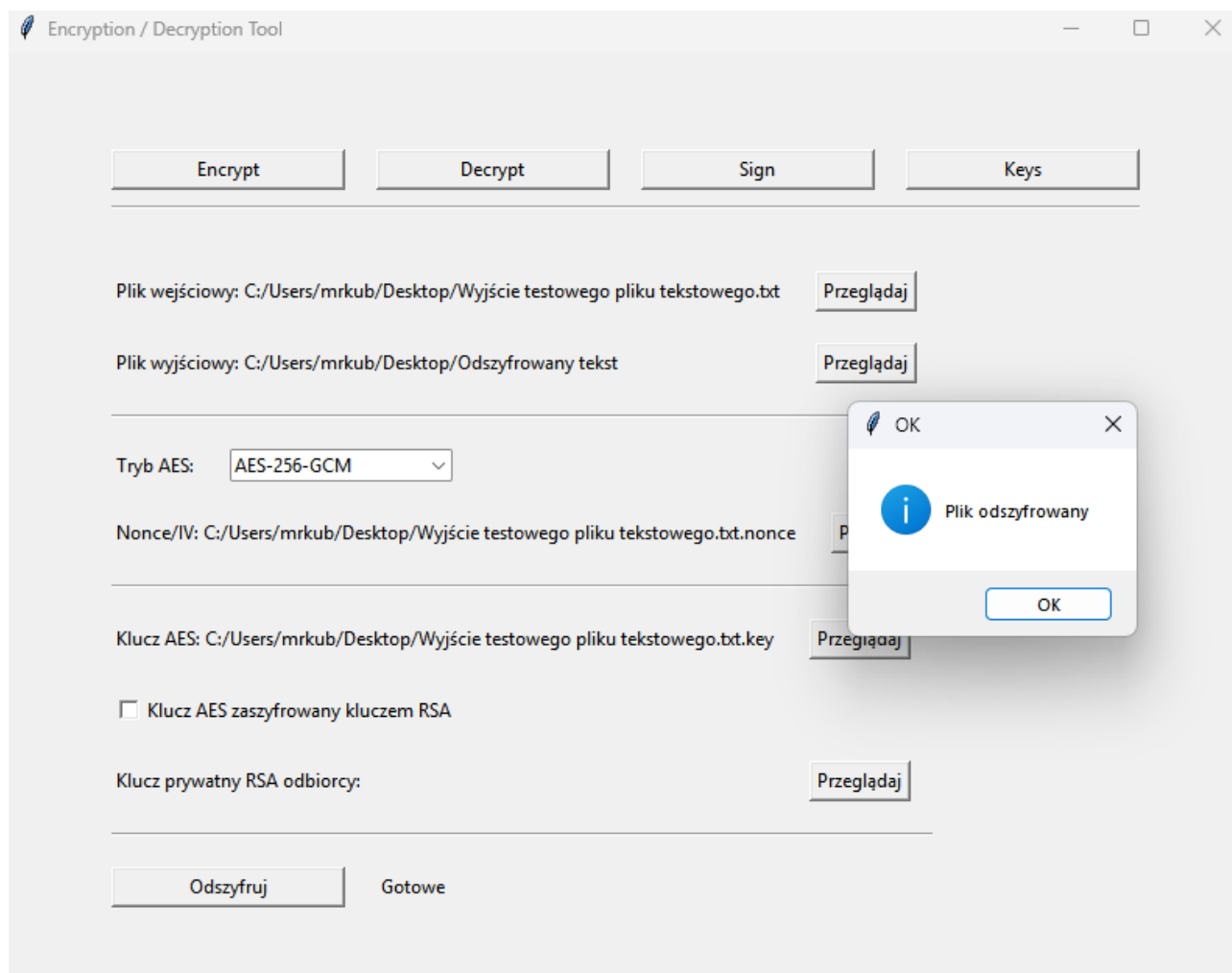


Rys. 7: Zaszyfrowanie pliku tekstowego.

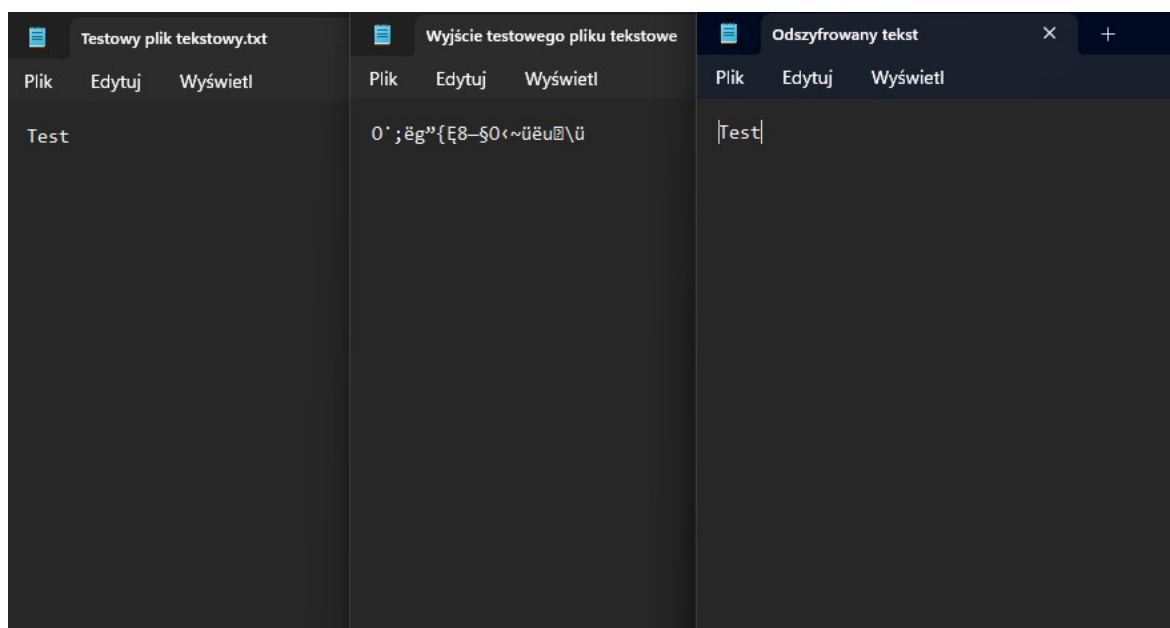
W wyniku działania programu utworzono:

- zaszyfrowany plik danych,
- plik zawierający klucz *AES*,
- plik zawierający wartość *Nonce*.

Następnie przeprowadzono proces odszyfrowania przy użyciu wygenerowanych wcześniej parametrów. Otrzymany plik był identyczny z oryginałem, co potwierdziło poprawność działania mechanizmu szyfrowania i deszyfrowania.



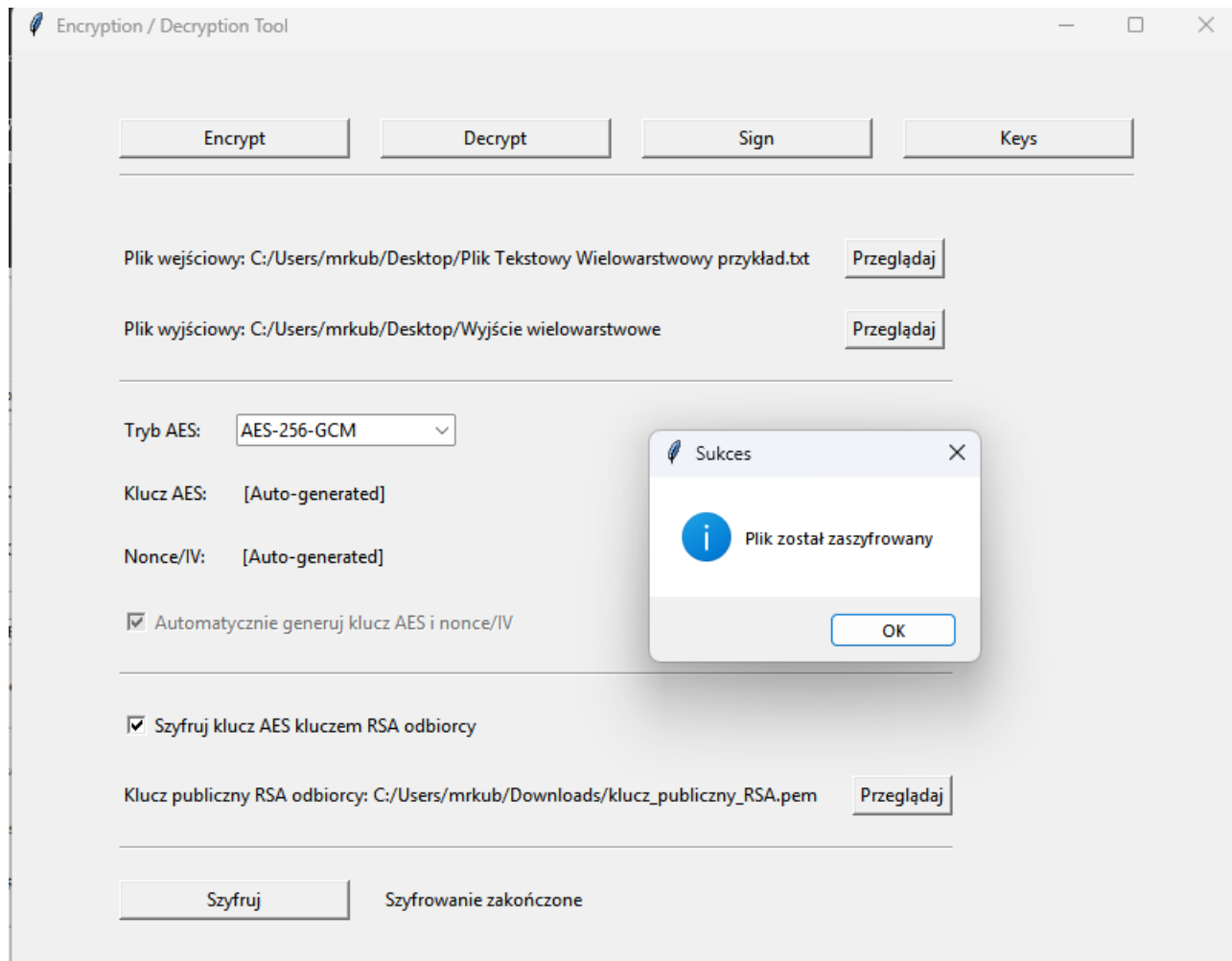
Rys. 8: Odszyfrowanie pliku tekstowego.



Rys. 9: Prezentacja procesu szyfrowania/odszyfrowania na podstawie plików.

8.3. Szyfrowanie hybrydowe

W drugim scenariuszu wykorzystano szyfrowanie hybrydowe. Dane zostały zaszyfrowane przy użyciu algorytmu *AES-256-GCM*, natomiast klucz AES został dodatkowo zabezpieczony przy użyciu klucza publicznego *RSA* odbiorcy.

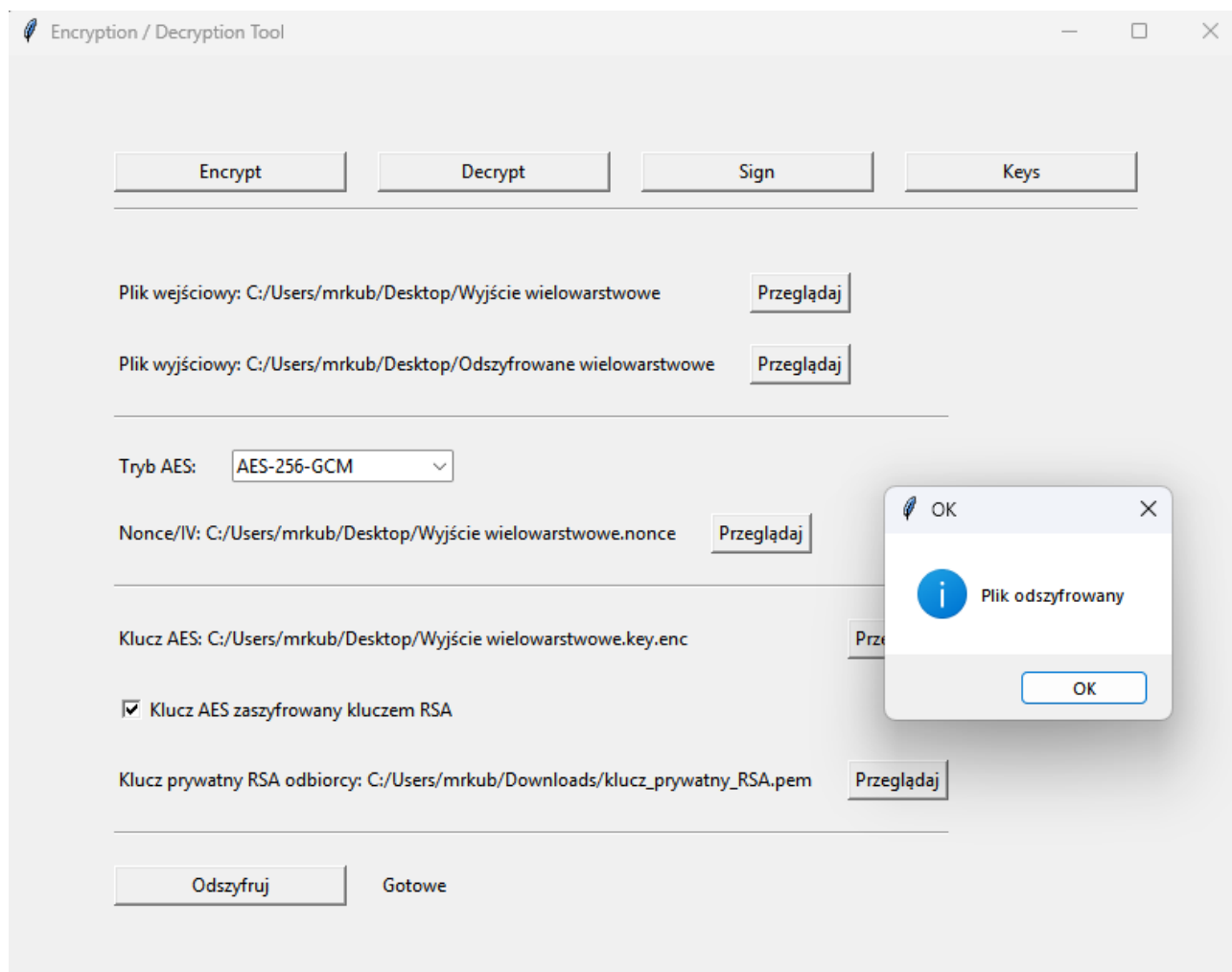


Rys. 10: Przykład szyfrowania wielowarstwowego przy użyciu klucza publicznego RSA.

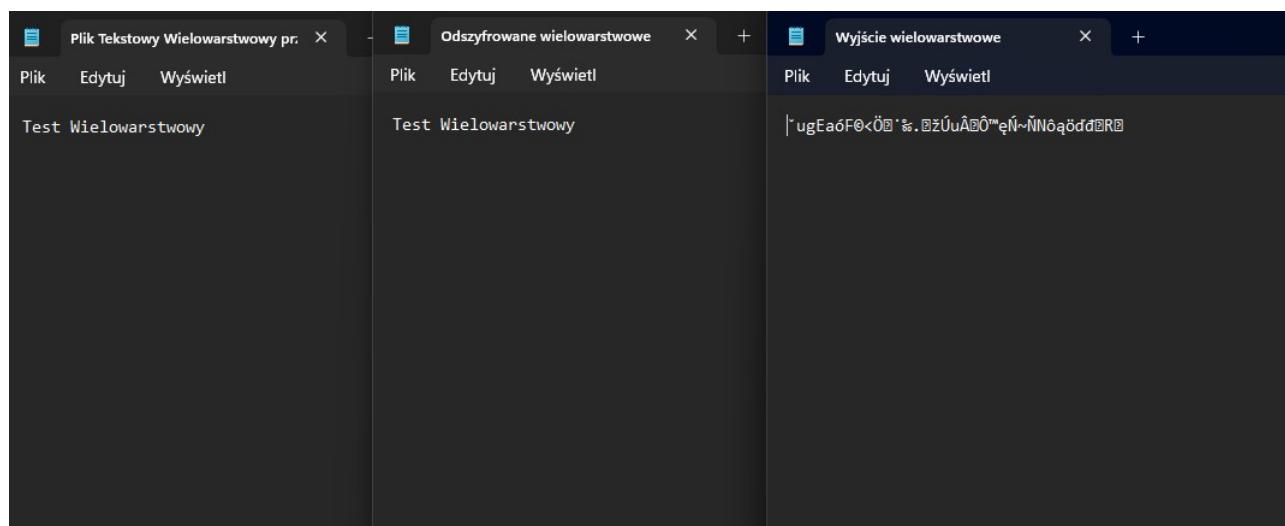
W celu odszyfrowania pliku wykorzystano:

- klucz prywatnego *RSA*,
- zaszyfrowany klucz *AES*,
- plik Nonce,
- zaszyfrowane dane.

Test wykazał poprawne odzyskanie danych oraz skuteczne zabezpieczenie klucza szyfrującego przed dostępem osób nieuprawnionych.



Rys. 11: Odszyfrowanie pliku tekstowego przy pomocy klucza prywatnego.



Rys. 12: Odszyfrowanie pliku tekstowego przy pomocy klucza prywatnego.

8.4. Podpisanie dokumentu i weryfikacja podpisu

W celu sprawdzenia poprawności działania podpisu cyfrowego podpisano dokument tekstowy przy użyciu klucza prywatnego RSA.

Plik do podpisania: C:/Users/mrkub/Downloads/Plik do podpisu.txt Przeglądaj

Klucz prywatny RSA nadawcy: C:/Users/mrkub/Downloads/klucz_prywatny_RSA.pem Przeglądaj

Plik podpisu: C:/Users/mrkub/Downloads/Podpisany plik Przeglądaj

Podpisz Plik podpisany

Plik do weryfikacji: C:/Users/mrkub/Downloads/Plik do podpisu.txt Przeglądaj

Klucz publiczny RSA nadawcy: C:/Users/mrkub/Downloads/klucz_publiczny_RSA_NOWY.pem Przeglądaj

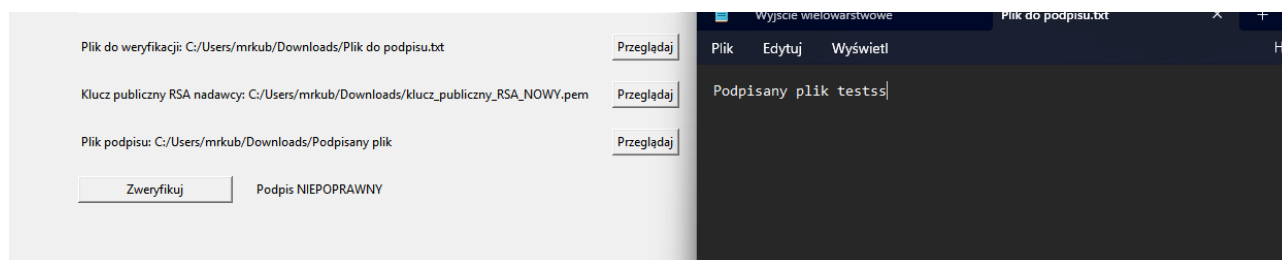
Plik podpisu: C:/Users/mrkub/Downloads/Podpisany plik Przeglądaj

Zweryfikuj Podpis poprawny

Rys. 13: Podpisanie pliku, oraz jego weryfikacja.

Po wygenerowaniu podpisu wykonano jego weryfikację z wykorzystaniem odpowiadającego klucza publicznego. Program poprawnie potwierdził autentyczność dokumentu.

Dodatkowo zmodyfikowano zawartość podpisanego pliku poprzez dopisanie pojedynczego znaku. Po ponownej weryfikacji aplikacja zgłosiła niepoprawny podpis, co potwierdziło skuteczność mechanizmu kontroli integralności danych.

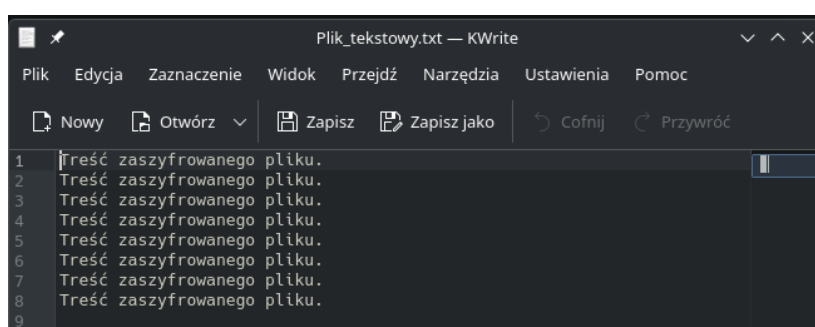


Rys. 14: Weryfikacja poprawności mechanizmu kontroli integralności danych.

8.5. Kompatybilność z innymi programami

Kompatybilność rozwiązania, a tym samym potwierdzenie działania oprogramowania zgodnie z założonym algorytmem, zweryfikowano przy użyciu narzędzia online *CyberChef*.

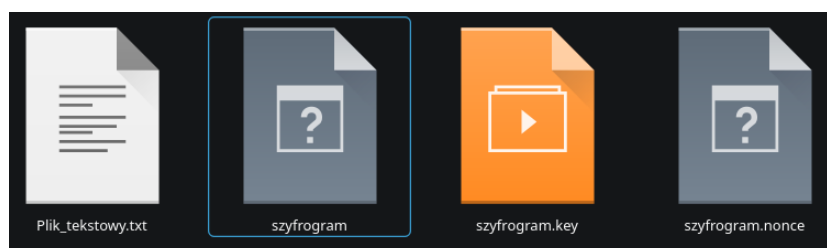
Ze względu na różnice w sposobie realizacji, głównie dotyczące formatów plików, konieczne było wcześniejsze przygotowanie danych wejściowych. W projekcie pliki zapisywane są w postaci binarnej, natomiast *CyberChef* wymaga danych w formacie szesnastkowym — lub innym obsługiwanym formacie tekstowym. W związku z tym wygenerowane pliki zostały najpierw przekonwertowane do postaci *hex*. Treść pliku przeznaczonego do zaszyfrowania przedstawiono na poniższym rysunku.



Rys. 15: Zawartość pliku zaszyfrowanego przez zaprojektowany program.

Na kolejnym rysunku przedstawiono pliki wygenerowane przez zaprojektowany program szyfrujący:

- „*Plik_tekstowy.txt*” – pierwotny plik zawierający treść do zaszyfrowania,
- „*szyfrogram*” – plik zawierający zaszyfrowane dane,
- „*szyfrogram.key*” – plik zawierający klucz AES,
- „*szyfrogram.nonce*” – plik zawierający wartość nonce wymaganą do odszyfrowania danych.



Rys. 16: Pliki wygenerowane poprzez procedurę szyfrowania.

Zawartość plików po konwersji do formatu szesnastkowego przedstawiono w tabeli poniżej.

Tabela 1. Dane po konwersji do formatu .hex.

Plik	Wartość po konwersji na typ .hex
szyfrogram	87610adbd87a68fce770c4f8610598225e49da589 45ef9ef1ae56847bf29d731dba9914a8c79333ad8 937f3516f6ee022b06c446d0ee559ab1fbadd4b0d de768c4aed122ac31fa447e84366911c91cfc086cf e842e1e5903a8d741012a51d6b9f0f580cdfa87ce 52faf201e667e77bb787217d32cf910153101023e c918a8000bb0fca2bb7553b2bae5019947af25d55 3bb60bf0c9157ab812e8876e4dd223d80b48a32a a966db7f048276f2be03836ebc2962734bdeabf6c 29e90827c2465bf7ec593baaf7cb9500e61225982 b1e3b6195df8630fb7eee0f16a73a29bb0abd5129 72f280a0113456c858e5741d8d7819cebf07d1c72 82a2adf1810e85180ad2
szyfrogram.nonce	2cf4262141121bc6f5bb5e6f
szyfrogram.key	9676e663d6a2c217bab5598af5487b772d2ca99c 4380fb1aa20eb7aaf2dc2fce

W celu poprawnego odszyfrowania danych *CyberChef* wymaga również podania wartości *GCM Tag*, generowanej przez algorytm *AES* pracujący w trybie *GCM*. W zaprojektowanym rozwiązaniu znacznik ten jest dołączany na końcu szyfrogramu jako ostatnie 16 bajtów. Z tego powodu przed wprowadzeniem danych do *CyberChef* konieczne było oddzielenie właściwego szyfrogramu od wartości *GCM Tag*.

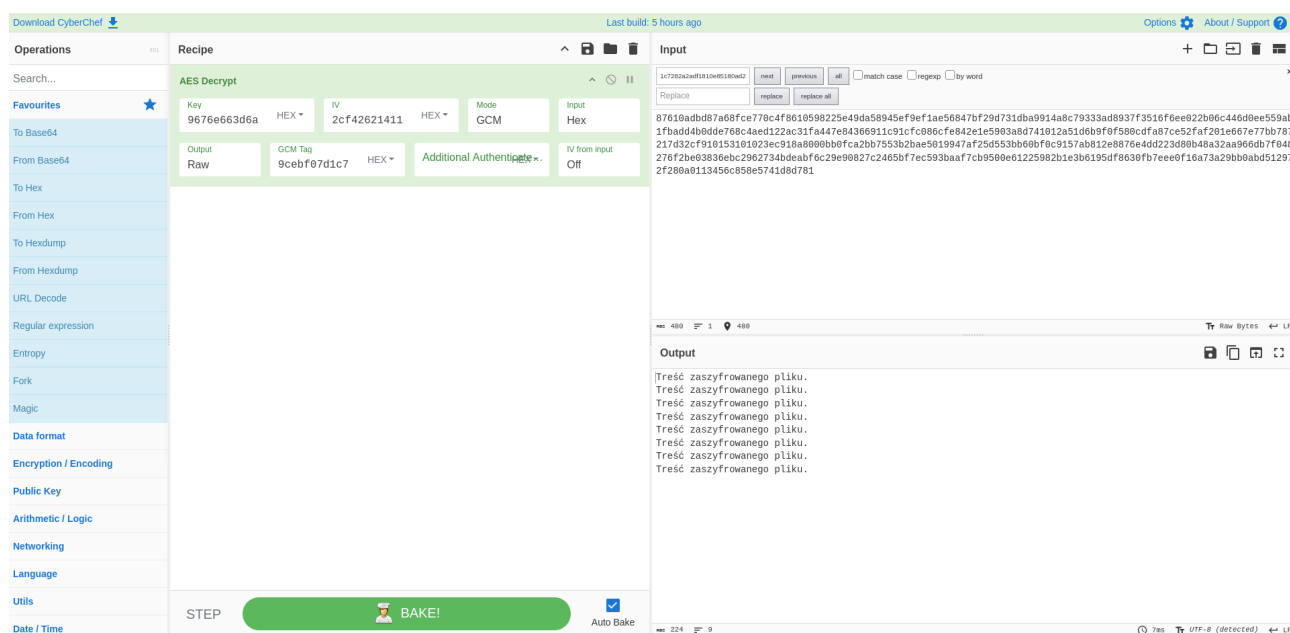
Końcowe dane wprowadzone do programu *CyberChef* miały postać przedstawioną w tabeli poniżej:

Tabela 2. Dane wejściowe dla programu *CyberChef*.

Parametr	Wartość
Key	9676e663d6a2c217bab5598af5487b772d2ca99c 4380fb1aa20eb7aaf2dc2fce
IV/Nonce	2cf4262141121bc6f5bb5e6f
GCM Tag	9cebf07d1c7282a2adf1810e85180ad2
Input	87610adbd87a68fce770c4f8610598225e49da589 45ef9ef1ae56847bf29d731dba9914a8c79333ad8 937f3516f6ee022b06c446d0ee559ab1fbadd4b0d de768c4aed122ac31fa447e84366911c91cfc086cf e842e1e5903a8d741012a51d6b9f0f580cdfa87ce 52faf201e667e77bb787217d32cf910153101023e c918a8000bb0fca2bb7553b2bae5019947af25d55

	3bb60bf0c9157ab812e8876e4dd223d80b48a32a a966db7f048276f2be03836ebc2962734bdeabf6c 29e90827c2465bf7ec593baaf7cb9500e61225982 b1e3b6195df8630fb7eee0f16a73a29bb0abd5129 72f280a0113456c858e5741d8d781
--	--

Wynik działania programu *CyberChef* przedstawiono na poniższym rysunku. Uzyskany rezultat potwierdza poprawność działania zaimplementowanego rozwiązania - odszyfrowana zawartość odpowiada pierwotnej treści pliku wejściowego zaszyfrowanego przy pomocy zaproponowanego programu.



Rys. 17: Wynik działania programu *CyberChef* na plikach wejściowych wygenerowanych przez zaprojektowane rozwiązanie.

8.6. Ocena poprawności działania systemu

Na podstawie przeprowadzonych testów stwierdzono, że aplikacja prawidłowo realizuje wszystkie założone funkcjonalności:

- szyfrowanie danych,
- odszyfrowywanie danych,
- generowanie kluczy RSA,
- tworzenie podpisów cyfrowych,
- weryfikację podpisów cyfrowych,
- szyfrowanie hybrydowe.

Nie stwierdzono utraty danych ani błędów wpływających na bezpieczeństwo działania systemu. Program poprawnie reagował również na sytuacje błędne, takie jak brak wymaganych plików lub użycie nieprawidłowego klucza. Uzyskane wyniki potwierdzają, że opracowane

rozwiązanie spełnia założenia projektowe i może być wykorzystywane do demonstracji praktycznych zastosowań kryptografii symetrycznej i asymetrycznej.

9. Podsumowanie i wnioski

Zrealizowany projekt aplikacji do szyfrowania plików i bezpiecznej wymiany danych spełnia założone cele funkcjonalne oraz projektowe. Udało się zaprojektować i zaimplementować system umożliwiający szyfrowanie oraz odszyfrowywanie danych, generowanie kluczy RSA, tworzenie i weryfikację podpisów cyfrowych, a także zastosowanie szyfrowania hybrydowego. Aplikacja została wykonana w języku *Python* z wykorzystaniem technologii opisanych w rozdziale 4, a jej modułowa budowa zwiększa czytelność kodu i ułatwia dalszy rozwój.

Przeprowadzone testy potwierdziły poprawność działania aplikacji. Program prawidłowo szyfruje i odszyfrowuje pliki, generuje pary kluczy RSA, obsługuje podpisy cyfrowe oraz wykrywa naruszenie integralności danych po modyfikacji pliku. Dodatkowa weryfikacja z wykorzystaniem narzędzia *CyberChef* potwierdziła zgodność działania zaimplementowanego szyfrowania AES-GCM.

Obecna wersja aplikacji dobrze sprawdza się jako narzędzie demonstracyjne pokazujące praktyczne zastosowanie mechanizmów kryptograficznych. Jednocześnie analiza projektu pozwala wskazać możliwe kierunki dalszego rozwoju systemu w postaci dodania obsługi większej liczby algorytmów kryptograficznych, czy automatyzacji zarządzania kluczami publicznymi użytkowników.

Na podstawie zbadanych zagrożeń i metod zapobiegania zagrożeniom można stwierdzić, że połączenie kryptografii symetrycznej i asymetrycznej jest skutecznym rozwiązaniem w bezpiecznej wymianie plików przy odpowiednim przeszkoleniu personelu, który będzie się posługiwał zadysponowanymi.

Poprawne wykorzystanie mechanizmów kryptograficznych pozwala zwiększyć poufność, integralność i autentyczność danych. Należy jednak pamiętać, że bezpieczeństwo systemu zależy nie tylko od zastosowanych algorytmów, ale także od właściwego zarządzania kluczami oraz świadomego korzystania z aplikacji przez użytkownika.

10. Bibliografia

[1] Advanced Encryption Standard - Wikipedia

https://pl.wikipedia.org/wiki/Advanced_Encryption_Standard

[2] SHA-2 – Wikipedia

<https://pl.wikipedia.org/wiki/SHA-2>

[3] Public-key cryptography – Wikipedia

https://en.wikipedia.org/wiki/Public-key_cryptography

[4] RSA Cryptosystem – Wikipedia

https://en.wikipedia.org/wiki/RSA_cryptosystem

[5] Dokumentacja biblioteki Tkinter – Python.org

<https://docs.python.org/3/library/tkinter.html>

[6] Dokumentacja biblioteki Cryptography – Cryptography.io

<https://cryptography.io/en/latest/>

[7] CyberChef – GitHub.com

<https://gchq.github.io/CyberChef>